E.L.I.S.A.

Entertainment Lookup with Information-State Architecture

Andreas Bartsiokas - Dialogue Systems 2 - FLoV, University of Gothenburg

5 November 2026

1. Introduction

Elisa is a spoken assistant that helps you pick a movie to watch. It is built on SISU, the information state update (ISU) approach combined with xstate statecharts and LLMs, with speechstate for speech and embeddings for retrieval.

A conversation has two phases. **Phase 1** is a settings form: who is watching, the youngest viewer's age, a time limit, genres to skip, the country and your streaming services. These are hard filters, and together they decide tonight's pool of movies. **Phase 2** is a spoken chat with Elisa about what you are in the mood for. She suggests movies from the pool until you take one, then wraps up.

Phase 1 is a form on purpose: the app depends on the filters gathered in phase 1 (One wrong assumption about a filter and magically all of the movies a user might wanna watch are filtered out). So it is safer (and faster), to keep those stricly in the form filling. The conversation is then free to build a taste profile in Phase 2, from vague, changing descriptions.

2. Data: TMDB and the local catalogue

TMDB is a free movie database with genres, runtimes, ratings, keywords, cast and streaming availability. Elisa uses it only before the chat starts.

The hard filters become a `/discover/movie` query: genres to skip, runtime, a US rating ceiling from the youngest viewer's age, and subscription services in your country. Movies under 60 minutes or with fewer than 100 votes are always left out. Runtime and rating are checked again locally, because TMDB's runtime filter is loose and an unrated movie must never pass an age limit.

A **local catalogue** keeps about 23,000 eligible movies with their details and a precomputed vector each. Without streaming services, the pool comes from the catalogue plus a short walk over movies released since the last top-up. With streaming services it comes from TMDB, because what is on which service changes every week. A typical pool holds 500 to 1,300 movies.

3. From moods to wishes

The first version asked for a mood, and a lookup table mapped it to genres ("goofy" to comedy). Genres were a hard TMDB filter, and about 60 popular results were re-ranked by embedding their descriptions live. A wrong genre guess hid the right movies, and no ranking could bring them back. Slot-by-slot NLU was fragile too.

The current version filters only on Phase 1 and ranks the **whole** pool by meaning. The NLU no longer fills slots. It rewrites what the user says into a **wish**, one short description of the movie they want ("a funny family adventure"), plus a list of things to **avoid**. Only the wish and the avoid texts are embedded at chat time.

4. Architecture

Part	Role
speechstate	Azure speech recognition and synthesis, with strict turn-taking
xstate	The top-level machine (form, pool, chat) and the dialogue manager as a child actor
SISU	The information state, moves, and update rules as the only thing that changes the state
NLU	An LLM turns an utterance into moves: wish, avoid, person, seen, reject, accept
NLG	An LLM writes Elisa's reply. The code picks the reply mode
Embeddings	Turn the wish into a vector and rank the pool
TMDB	The pool and the facts Elisa may use, so she cannot invent a plot

SISU and the information state. The state is plain JSON. Next to the usual private and shared parts, it holds the domain: the wish, the avoid list, wanted or avoided people, the ranked candidates, and the movies suggested, rejected and accepted. The Phase 1 settings sit in it read only. Moves are the only way in. The LLM never writes the state. It returns moves, and update rules integrate them. Elisa's own moves are integrated too, which is how her suggestion becomes shared. A question under discussion (`qud`) records what her last turn left open (a taste question, a suggestion, or nothing).

xstate. The top-level machine runs Prepare, Intake, FindingMovies and Main. Main has three parallel regions: listening, speaking, and the dialogue move engine (DME). They talk only through events, so the DME does not know speech exists, and a typed chat in the terminal can drive the same DME. The DME alternates Select (say something) and Update (hear, integrate, retrieve), as in the SISU design. Each update rule is wired in as a guard and an action, and a chain of states fires every rule that applies, so several moves from one utterance are all integrated. LLM calls and embeddings are promise actors, and each failure has its own state, so a failed call never stops the dialogue.

speechstate wraps Azure Speech as an xstate actor: US English, a 7 second no-input timeout and no barge-in. Elisa finishes speaking, then she listens. Silence is a move of its own and gets a fixed line with no LLM call.

5. Understanding: NLU with grounding

The NLU makes one LLM call per turn at temperature 0. It sees the last 6 turns, a summary of the state, the last suggestion, and a prompt block chosen by the `qud`, and returns moves. The wish is rewritten whole each time ("a light romantic comedy, but funnier"), so it can be embedded as one description.

Plain code then checks the output:

- **Refusals are split out of the wish.** "Something magical, but no princesses" becomes the wish "something magical" and the avoid "princesses". An embedding cannot tell "no princesses" from "princesses", so a refusal left in the wish would pull princess movies up. Descriptive negations such as "non-stop action" stay in the wish.
- **Grounding** drops moves the user's own words do not support: a person they did not name, a wish or avoid copied from Elisa's last turn, a hedge like "not sure yet" taken as taste, and an accept when nothing was suggested. Small models often copy Elisa's words into the user's moves, so this check matters.
- **A bare "yes"** right after a suggestion is an accept, with no LLM call. Anything longer ("yes, but something shorter") goes to the LLM.

6. Retrieval: embeddings and ranking

Retrieval is a small RAG. Movies are embedded with `qwen3-embedding:4b` (2,560 dimensions) on local Ollama, the strongest of the models compared. A movie's text is its title, year, genres, overview

and up to 15 TMDB keywords.

Every turn, the whole pool is ranked in memory:

```
score = 0.8*semantic + 0.1*popularity - avoid - reject + person
```

`semantic` is the cosine similarity between the wish and the movie, and `popularity` the vote count, both as percentiles among the eligible movies. Raw cosine similarities sit in a narrow band that moves with every query, so fixed weights on them would mean something different each turn. On percentiles every term runs from 0 to 1. The `avoid` penalty reaches only the 5% most similar to an avoid text, so "no princesses" sinks princess movies without pulling down every fairy tale. A rejected movie sinks its closest 10% with a smaller weight, so "not that one" also sinks its near twins, while "already seen" removes only the movie itself. A wanted actor or director adds 1 and puts their movies first. The best 8 movies, with their TMDB facts, go to the NLG as the only movies Elisa may suggest.

7. LLM backends and NLG

The backends are OpenAI-compatible endpoints in JSON mode, tried in order until one answers. The 4B embedding model fills the local GPU, so the **NLU** tries the university's llama3.1 first, then Groq, then a local qwen2.5:7b as a last resort. The **NLG** prefers Groq's gpt-oss-120b for its better replies, then llama3.1, then the local model. Groq has a daily limit and 8,000 tokens per minute, so it is the NLU's fallback and only the NLG's first choice.

The NLG runs at temperature 0.7. The code, not the LLM, picks the reply mode from the state: `explore` (ask about taste), `recommend` (suggest a candidate, once there is a wish or a wanted person and the user has spoken at least 3 times), `discuss` (talk about the open suggestion) and `wrapup` (after an accept). Each mode has its own prompt. Candidates are shown only in recommend mode, a suggested movie must be one of them, and a reply that names a movie too early is retried once and then replaced by a fixed line.

A sample dialogue (illustrative, family profile, youngest viewer 8, 100 minutes):

Turn	What the system does
Elisa: Hi Sam! What are you all in the mood for tonight?	Fixed opening, no LLM
You: Something funny for the kids, but no princesses.	wish: a funny family movie, avoid: princesses
Elisa: How about Chicken Run? A clever escape comedy, 84 minutes.	Pool re-ranked, top candidate suggested
You: Not that one. Something with more adventure.	reject: Chicken Run, wish rewritten whole
Elisa: Then try Wallace & Gromit: The Curse of the Were-Rabbit.	Look-alikes of Chicken Run sink
You: Yes, let's watch that!	A clear yes becomes accept, no LLM call

8. Evaluation

- `nlu-bench`: 32 cases from my own test chats (wishes, avoids, people, rejections, accepts, negated tails). The university llama3.1 passes 30 to 31 of 32, and the misses vary between runs.
- `rank-bench`: hand-labelled wishes on two real pools, scored by recall@10 and by forbidden titles in the top 10. Recall@10 is 52% (61 of 118) with 1 forbidden title (Shrek for "magical, but no princesses"). Widening the avoid penalty to 10% removed Shrek but cost recall, so it stays at 5%.
- **Live typed chats** with three profiles (solo, couple, family) check that a whole conversation ends in a sensible pick.

9. Known issues and limits

These lists hold the most important points and are not exhaustive. More are tracked on the project's GitHub issues tab.

Issues

1. The NLU can still miss or blur a wish, mostly when a new taste comes right after a suggestion ("funny with animals").
2. Small models echo Elisa's words as the user's. Grounding catches most cases, but it is rule-based and can miss a paraphrase.
3. Groq's limits can sometimes make the conversation unstable, when the per-minute limit is hit and a fallback model takes over.

Limits

1. English only: the chat, the prompts and the speech.
2. Age limits use US ratings for everyone, so movies without a US rating drop out of age-limited pools.
3. Year and runtime are not part of the chat. Changing them means going back to the form.

10. Future work

The architecture is in place. What remains is mostly about making Elisa better at the conversation itself.

- **User testing, first of all.** The benches come from my own chats. Sessions with real viewers would show which wishes the NLU misses, whether 52% recall is good enough in practice, and how long people are willing to talk before taking a movie.
- **Barge-in.** Turn-taking is strict, so you cannot cut into a long reply. speechstate has a barge-in setting, but it is off and untested here, and the turn logic in the top-level machine assumes one speaker at a time.
- **Memory between sessions.** Only the Phase 1 profile is kept today. Remembering past likes and rejections under the same name would let Elisa skip movies you have seen and start closer to your taste. This needs a design first, since remembered taste must not override what you ask for tonight.
- **Saying what was understood.** When grounding changes a move, for example a refusal split off the wish, Elisa could confirm it ("Funny with animals, got it.") as explicit feedback, so a wrong reading is caught early.
- **Stronger handling of actors and directors.** "A Nolan movie" sometimes becomes a wish rather than a person move, which loses the exact credit match.
- **Year and runtime in the chat.** "Something more recent" or "shorter" could become soft ranking terms instead of a trip back to the form.
- **Better models.** A larger or fine-tuned NLU would need fewer code repairs, and an NLG model tuned for chat rather than tasks may sound more natural.